

핸즈온 랩: 데이터 웨어하우스에서 데이터 분석하기

Microsoft Fabric에서 **data warehouse**는 대규모 분석을 위한 관계형 데이터베이스를 제공합니다.

개념 설명: **Data Warehouse**와 **Lakehouse SQL endpoint**의 가장 큰 차이점은 데이터 조작 능력에 있습니다.

Lakehouse의 SQL 엔드포인트는 주로 Delta Lake 테이블에 대한 **읽기 전용(read-only)** 쿼리에 최적화되어 있습니다. 반면, Data Warehouse는 테이블의 데이터를 삽입(INSERT), 업데이트(UPDATE), 삭제(DELETE)하는 능력을 포함하여 완전한 SQL 의미 체계(semantics)를 제공하는 **읽기-쓰기(read-write)** 환경입니다. 이는 전통적인 관계형 데이터베이스 관리 시스템 (RDBMS)과 매우 유사한 경험을 제공합니다.

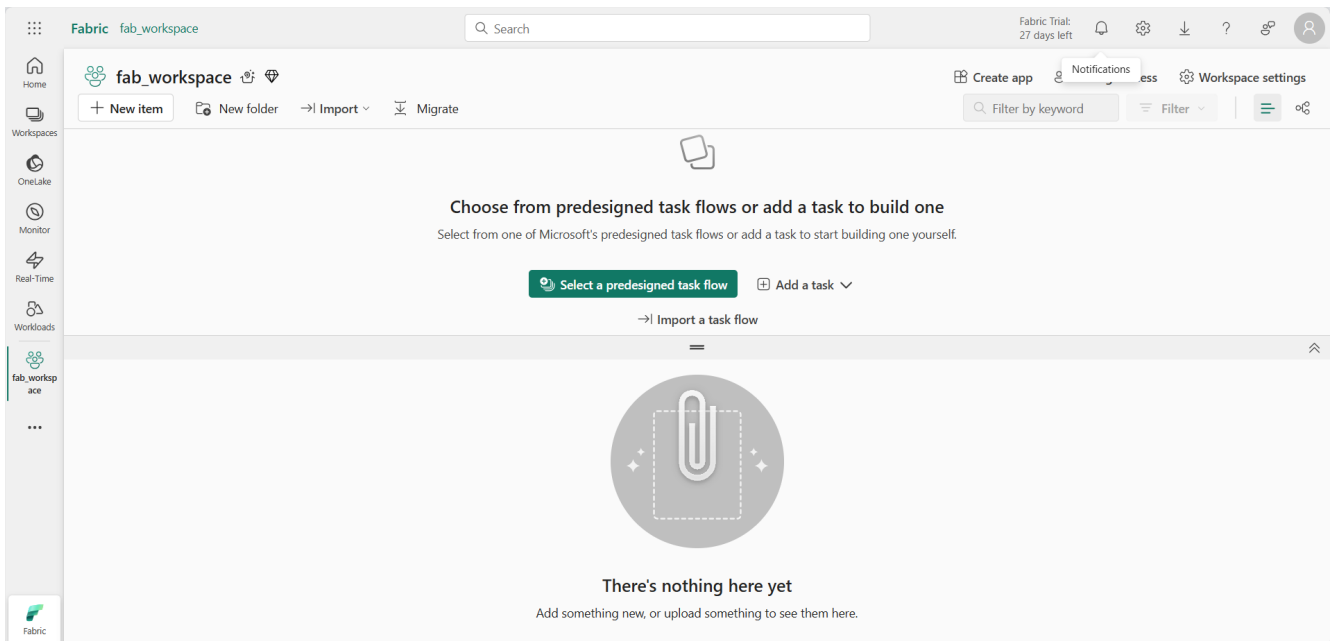
이 실습을 완료하는 데 약 **30분**이 소요됩니다.

참고: 이 실습을 완료하려면 [Microsoft Fabric 평가판](#)이 필요합니다.

Workspace 만들기

Fabric에서 데이터 작업을 시작하기 전에 Fabric 평가판이 활성화된 Workspace(작업 영역)를 만들어야 합니다.

1. 웹 브라우저에서 [Microsoft Fabric 홈페이지](https://app.fabric.microsoft.com/home?experience=fabric) `https://app.fabric.microsoft.com/home?experience=fabric` 로 이동하여 Fabric 자격 증명으로 로그인합니다.
2. 왼쪽 메뉴 모음에서 **Workspaces** 아이콘()을 선택합니다.
3. 원하는 이름으로 새 Workspace를 만듭니다. **Advanced** 섹션에서 Fabric 용량(*Trial, Premium, 또는 Fabric*)을 포함하는 라이선스 모드를 선택해야 합니다.
4. 새 Workspace가 열리면 처음에는 비어 있어야 합니다.



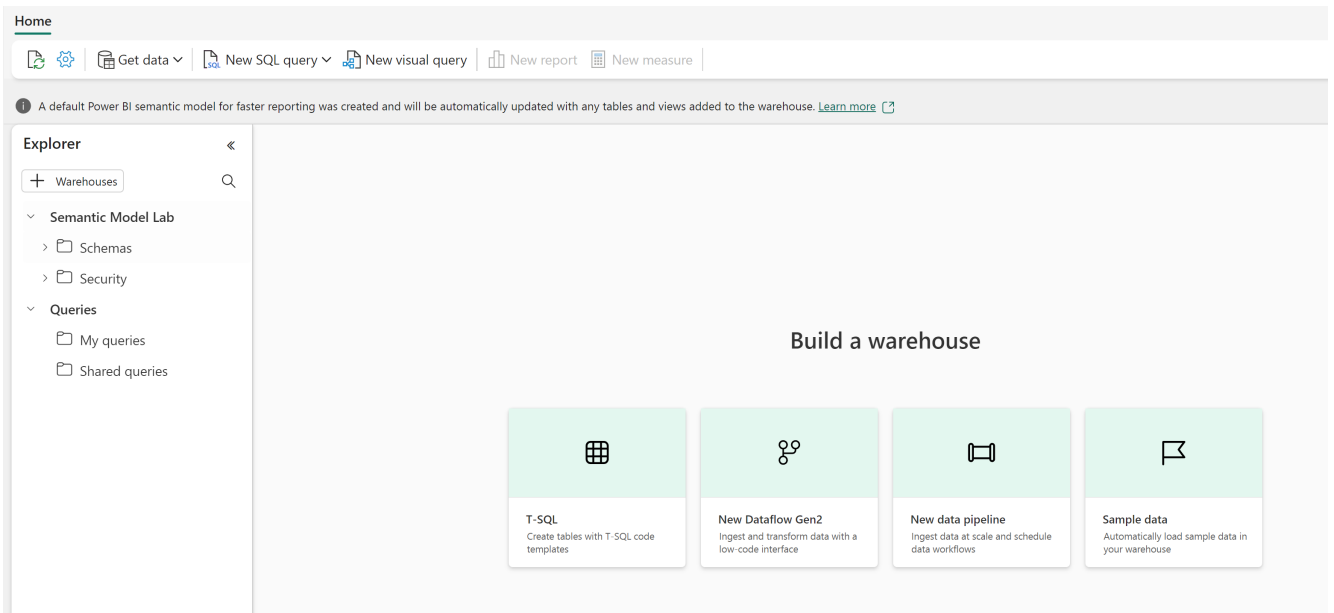
Data Warehouse 만들기

이제 Workspace가 준비되었으니, 데이터 웨어하우스를 만들 차례입니다.

1. 왼쪽 메뉴 모음에서 **Create**를 선택합니다. *New* 페이지의 *Data Warehouse* 섹션에서 **Warehouse**를 선택하고, 원하는 고유한 이름을 지정합니다.

참고: **Create** 옵션이 사이드바에 고정되어 있지 않다면, 먼저 줄임표(...) 옵션을 선택해야 합니다.

약 1분 후, 새로운 웨어하우스가 생성됩니다.



테이블 생성 및 데이터 삽입

웨어하우스는 테이블 및 기타 객체를 정의할 수 있는 관계형 데이터베이스입니다.

1. 새 웨어하우스에서 **T-SQL** 타일을 선택하고 다음 `CREATE TABLE` 문을 사용합니다.

```
CREATE TABLE dbo.DimProduct
(
    ProductKey INTEGER NOT NULL,
    ProductAltKey VARCHAR(25) NULL,
    ProductName VARCHAR(50) NOT NULL,
    Category VARCHAR(50) NULL,
    ListPrice DECIMAL(5,2) NULL
);
GO
```

코드 설명:

- `CREATE TABLE dbo.DimProduct`: `dbo` 스키마에 `DimProduct` 라는 이름의 새 테이블을 생성합니다. 데이터 웨어하우스에서는 종종 차원(Dimension) 테이블을 `Dim` 접두사로 시작합니다.
 - `ProductKey INTEGER NOT NULL`: 각 제품을 고유하게 식별하는 정수형 기본 키 열을 정의합니다. `NOT NULL` 은 이 열에 값이 반드시 있어야 함을 의미합니다.
 - `GO`: Transact-SQL(T-SQL)에서 배치(batch)의 끝을 나타내는 구분자입니다.
2. > **Run** 버튼을 사용하여 SQL 스크립트를 실행하면, 데이터 웨어하우스의 `dbo` 스키마에 **DimProduct**라는 새 테이블이 생성됩니다.
 3. 툴바의 **Refresh** 버튼을 사용하여 뷰를 새로 고칩니다. 그런 다음 **Explorer** 창에서 **Schemas > dbo > Tables**를 확장하여 **DimProduct** 테이블이 생성되었는지 확인합니다.
 4. **Home** 메뉴 탭에서 **New SQL Query** 버튼을 사용하여 새 쿼리를 만들고 다음 `INSERT` 문을 입력합니다.

```
INSERT INTO dbo.DimProduct
VALUES
(1, 'RING1', 'Bicycle bell', 'Accessories', 5.99),
(2, 'BRITE1', 'Front light', 'Accessories', 15.49),
(3, 'BRITE2', 'Rear light', 'Accessories', 15.49);
GO
```

5. 새 쿼리를 실행하여 **DimProduct** 테이블에 세 개의 행을 삽입합니다.
6. 쿼리가 완료되면 **Explorer** 창에서 **DimProduct** 테이블을 선택하고 세 개의 행이 테이블에 추가되었는지 확인합니다.
7. **Home** 메뉴 탭에서 **New SQL Query** 버튼을 사용하여 새 쿼리를 만듭니다. 그런 다음 <https://raw.githubusercontent.com/MicrosoftLearning/dp-data/main/create-dw.txt>의 Transact-SQL 코드를 복사하여 새 쿼리 창에 붙여넣습니다.
8. 쿼리를 실행합니다. 이 스크립트는 간단한 데이터 웨어하우스 스키마를 생성하고 일부 데이터를 로드합니다. 스크립트 실행에는 약 30초가 소요됩니다.
9. 툴바의 **Refresh** 버튼을 사용하여 뷰를 새로 고칩니다. 그런 다음 **Explorer** 창에서 데이터 웨어하우스의 **dbo** 스키마에 이제 다음 네 개의 테이블이 포함되어 있는지 확인합니다.
 - **DimCustomer**
 - **DimDate**
 - **DimProduct**
 - **FactSalesOrder**

데이터 모델 정의하기

개념 설명: 관계형 데이터 웨어하우스는 일반적으로 **Fact(사실)** 테이블과 **Dimension(차원)** 테이블로 구성됩니다. Fact 테이블은 비즈니스 성과를 분석하기 위해 집계할 수 있는 숫자 측정값(예: 판매 수익)을 포함하고, Dimension 테이블은 데이터를 집계할 수 있는 엔터티의 속성(예: 제품, 고객 또는 시간)을 포함합니다. 이 구조를 **스타 스키마(Star Schema)**라고 하며, 분석 쿼리에 최적화되어 있습니다.

핸즈온의 의미: 이 단계에서는 Fact 테이블과 Dimension 테이블 간의 관계(기본 키-외래 키)를 시각적으로 정의합니다. 이 모델링 작업을 통해 Power BI와 같은 BI 도구는 테이블 간의 관계를 자동으로 이해하고, 사용자가 코드를 작성하지 않고도 데이터를 직관적으로 슬라이스 앤 다이스(Slice and Dice)할 수 있게 됩니다.

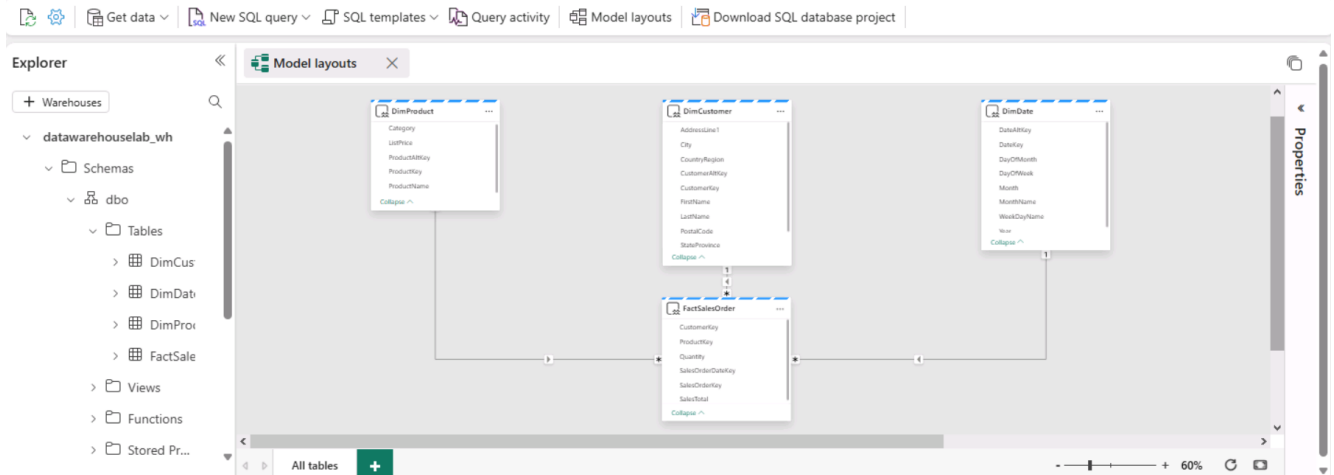
1. 툴바에서 **Model** 레이아웃 버튼을 선택합니다.
2. 모델 창에서 데이터 웨어하우스의 테이블을 재정렬하여 **FactSalesOrder** 테이블이 중앙에 오도록 배치합니다.
3. **FactSalesOrder** 테이블의 **ProductKey** 필드를 **DimProduct** 테이블의 **ProductKey** 필드로 끌어다 놓습니다. 그런 다음 다음 관계 세부 정보를 확인합니다.
 - **From table:** FactSalesOrder
 - **Column:** ProductKey
 - **To table:** DimProduct
 - **Column:** ProductKey
 - **Cardinality:** Many to one (*:1)
 - **Cross filter direction:** Single

- **Make this relationship active:** 선택됨
- **Assume referential integrity:** 선택되지 않음

4. 동일한 프로세스를 반복하여 다음 테이블 간에 다대일(many-to-one) 관계를 만듭니다.

- **FactSalesOrder.CustomerKey → DimCustomer.CustomerKey**
- **FactSalesOrder.SalesOrderDateKey → DimDate.DateKey**

모든 관계가 정의되면 모델은 다음과 같이 보여야 합니다.



데이터 웨어하우스 테이블 쿼리하기

데이터 웨어하우스는 관계형 데이터베이스이므로 SQL을 사용하여 테이블을 쿼리할 수 있습니다.

Fact 테이블과 Dimension 테이블 쿼리하기

관계형 데이터 웨어하우스의 대부분의 쿼리는 관련 테이블 간의 데이터를 집계하고 그룹화하는 작업(JOIN 절 사용)을 포함합니다.

1. 새 SQL 쿼리를 만들고 다음 코드를 실행합니다.

```
SELECT d.[Year] AS CalendarYear,
       d.[Month] AS MonthOfYear,
       d.MonthName AS MonthName,
       SUM(so.SalesTotal) AS SalesRevenue
FROM FactSalesOrder AS so
JOIN DimDate AS d ON so.SalesOrderDateKey = d.DateKey
GROUP BY d.[Year], d.[Month], d.MonthName
ORDER BY CalendarYear, MonthOfYear;
```

코드 설명:

- **JOIN DimDate ...:** FactSalesOrder 테이블과 DimDate 테이블을 날짜 키로 조인합니다.
- **SUM(so.SalesTotal):** 판매 사실 테이블의 SalesTotal 을 합산하여 총 매출을 계산합니다.
- **GROUP BY d.[Year], d.[Month], d.MonthName:** 날짜 차원 테이블의 연도, 월, 월 이름으로 그룹화하여 월별 매출을 집계합니다.

날짜 차원의 속성을 사용하면 Fact 테이블의 측정값을 여러 계층 수준(이 경우 연도 및 월)에서 집계할 수 있습니다. 이는 데이터 웨어하우스에서 일반적인 패턴입니다.

- 다음과 같이 쿼리를 수정하여 집계에 두 번째 차원을 추가합니다.

```
SELECT d.[Year] AS CalendarYear,
       d.[Month] AS MonthOfYear,
       d.MonthName AS MonthName,
       c.CountryRegion AS SalesRegion,
       SUM(so.SalesTotal) AS SalesRevenue
FROM FactSalesOrder AS so
JOIN DimDate AS d ON so.SalesOrderDateKey = d.DateKey
JOIN DimCustomer AS c ON so.CustomerKey = c.CustomerKey
GROUP BY d.[Year], d.[Month], d.MonthName, c.CountryRegion
ORDER BY CalendarYear, MonthOfYear, SalesRegion;
```

코드 설명: `JOIN DimCustomer...`가 추가되어 이제 고객 차원 테이블의 `CountryRegion` (판매 지역) 정보까지 사용하여 더 세분화된 분석(연도별, 월별, 지역별 매출)을 수행합니다.

- 수정된 쿼리를 실행하고 결과를 검토합니다.

뷰(View) 만들기

개념 설명: **View**는 하나 이상의 테이블에서 파생된 가상 테이블입니다. 복잡한 SQL 쿼리 로직을 뷰 안에 저장해두면, 사용자는 복잡한 쿼리를 반복해서 작성할 필요 없이 간단한 `SELECT` 문으로 뷰를 조회할 수 있습니다. 이는 쿼리를 단순화하고 보안을 강화하는 데 유용합니다.

- 이전에 만든 쿼리를 다음과 같이 수정하여 뷰를 만듭니다 (뷰를 만들려면 `ORDER BY` 절을 제거해야 함).

```
CREATE VIEW vSalesByRegion
AS
SELECT d.[Year] AS CalendarYear,
       d.[Month] AS MonthOfYear,
       d.MonthName AS MonthName,
       c.CountryRegion AS SalesRegion,
       SUM(so.SalesTotal) AS SalesRevenue
FROM FactSalesOrder AS so
JOIN DimDate AS d ON so.SalesOrderDateKey = d.DateKey
JOIN DimCustomer AS c ON so.CustomerKey = c.CustomerKey
GROUP BY d.[Year], d.[Month], d.MonthName, c.CountryRegion;
```

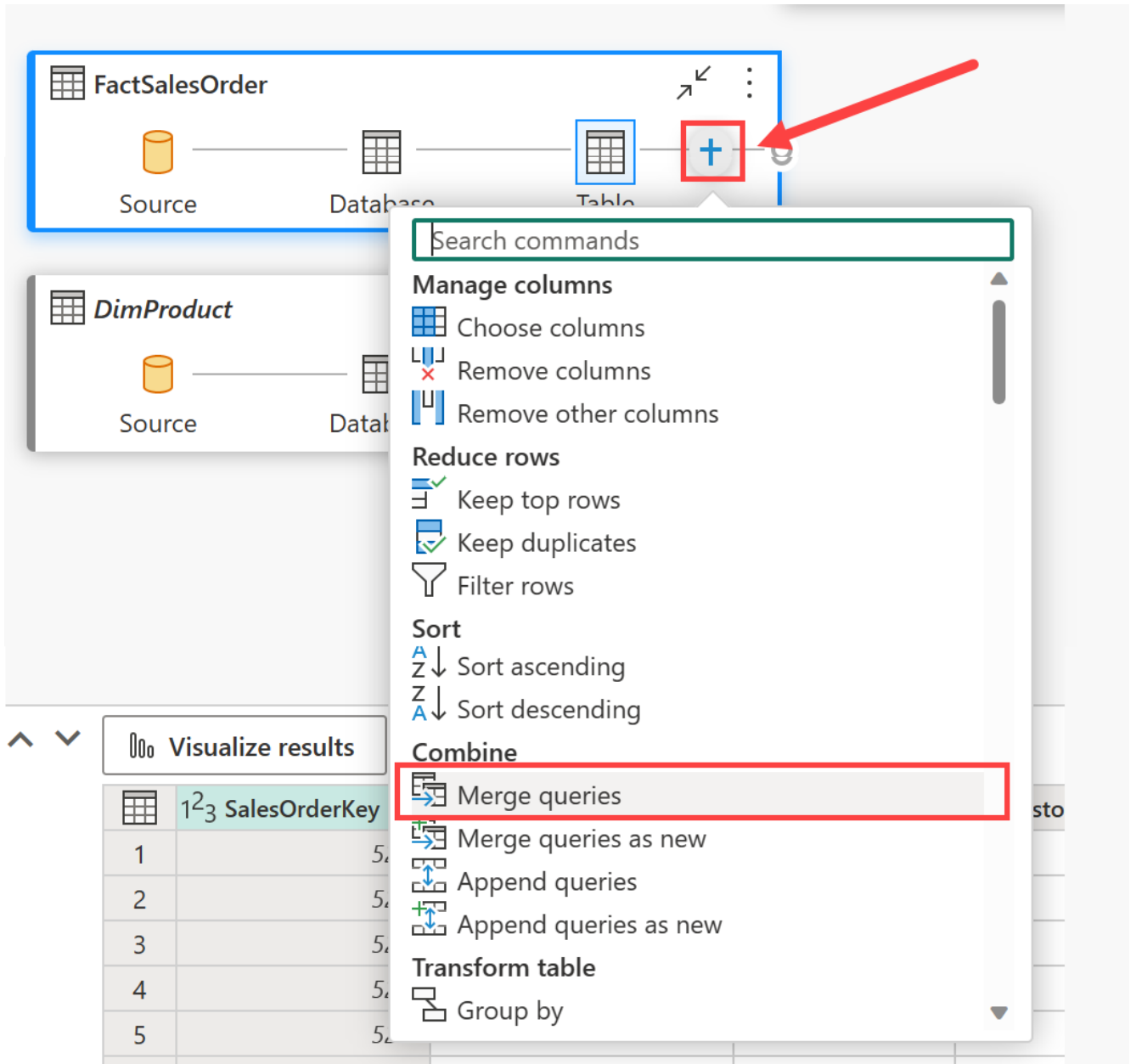
- 쿼리를 실행하여 뷰를 만듭니다. 그런 다음 데이터 웨어하우스 스키마를 새로 고치고 새 뷰가 **Explorer** 창에 나열되는지 확인합니다.
- 새 SQL 쿼리를 만들고 다음 `SELECT` 문을 실행합니다.

```
SELECT CalendarYear, MonthName, SalesRegion, SalesRevenue
FROM vSalesByRegion
ORDER BY CalendarYear, MonthOfYear, SalesRegion;
```

시각적 쿼리(Visual Query) 만들기

핸즈온의 의미: 이 단계는 Fabric의 유연성을 보여줍니다. 동일한 데이터 웨어하우스를 사용하는 상황에서, SQL 코드를 선호하는 사용자는 T-SQL 편집기를, Power BI나 Excel에 익숙하여 시각적 도구를 선호하는 사용자는 코드를 작성하지 않고도 시각적 쿼리 디자이너를 사용할 수 있습니다.

1. **Home** 메뉴에서 **New SQL query** 아래의 옵션을 확장하고 **New visual query**를 선택합니다.
2. **FactSalesOrder**를 캔버스로 끌어다 놓습니다. 아래의 **Preview** 창에 테이블 미리보기가 표시되는 것을 확인하세요.
3. **DimProduct**를 캔버스로 끌어다 놓습니다.
4. 캔버스에 있는 **FactSalesOrder** 테이블의 **(+)** 버튼을 사용하여 **Merge queries**를 선택합니다.



5. **Merge queries** 창에서 오른쪽 테이블로 **DimProduct**를 선택합니다. 두 쿼리에서 모두 **ProductKey**를 선택하고, 기본 **Left outer** 조인 유형을 그대로 두고 **OK**를 클릭합니다.
6. **Preview**에서 새 **DimProduct** 열이 **FactSalesOrder** 테이블에 추가된 것을 확인합니다. 열 이름 오른쪽의 화살표를 클릭하여 열을 확장하고, **ProductName**을 선택한 후 **OK**를 클릭합니다.

☒ (Select all)
☐ ProductKey
☐ ProductAltKey
☒ **ProductName**
☐ Category
☐ ListPrice

☐ Use original column name as prefix

ProductKey	Quantity	SalesTotal	DimProduct
550	10	22950	[Table]

- 이제 **ProductName** 열을 사용하여 **Cable Lock** 데이터만 보도록 쿼리의 데이터를 필터링할 수 있습니다.
- 여기서 **Visualize results** 또는 **Download Excel file**을 선택하여 이 단일 쿼리의 결과를 분석할 수 있습니다.

리소스 정리

이 실습에서는 여러 테이블을 포함하는 데이터 웨어하우스를 만들었습니다. SQL을 사용하여 테이블에 데이터를 삽입하고, T-SQL 및 시각적 쿼리 도구를 사용하여 테이블을 쿼리했습니다. 마지막으로 다운스트림 분석 및 보고를 위해 데이터 웨어하우스의 기본 데이터 세트에 대한 데이터 모델을 향상시켰습니다.

데이터 웨어하우스 탐색을 마쳤다면 이 실습을 위해 만든 Workspace를 삭제할 수 있습니다.

- 왼쪽 막대에서 Workspace 아이콘을 선택하여 포함된 모든 항목을 봅니다.
- 툴바의 ... 메뉴에서 **Workspace settings**를 선택합니다.
- General** 섹션에서 **Remove this workspace**를 선택합니다.